Project Part 3 — README

Links

- **GitHub Repository:** https://github.com/tymedina100/CSE464-2025-tmedina7
- **Pull Request (refactor branch → main):**
  https://github.com/tymedina100/CSE464-2025-tmedina7/pull/8

Commit Links

- **Refactoring #1:**
  https://github.com/tymedina100/CSE464-2025-tmedina7/pull/8/commits/3698983db8daeb844329f67fe8b9244ae3115224

- **Refactoring #2:**
  https://github.com/tymedina100/CSE464-2025-tmedina7/pull/8/commits/46ec1204d45e396b4ed7cbd7d0fb2f2b1a109e88

- **Refactoring #3:**
  https://github.com/tymedina100/CSE464-2025-tmedina7/pull/8/commits/90de949d531e077e16a2f99283039bcb6c70345c

- **Refactoring #4:**
  https://github.com/tymedina100/CSE464-2025-tmedina7/pull/8/commits/dfb93a28bec8335d101d8dc9b9a5fce2f8aa348f

- **Refactoring #5:**
  https://github.com/tymedina100/CSE464-2025-tmedina7/pull/8/commits/bb0066ffc6e88119667d052ddd032796fe3773f1

- **Template Method implementation (BFS/DFS):**
  https://github.com/tymedina100/CSE464-2025-tmedina7/pull/8/commits/fc36edb59afe48c0b31e3d21d1ba3ffa19afec0d

- **Strategy selection via algo parameter:**
  https://github.com/tymedina100/CSE464-2025-tmedina7/pull/8/commits/4095e0cd1f9a721482d97fe9ba187ec531c23233

- **Random Walk (implemented using Template + Strategy):**
  https://github.com/tymedina100/CSE464-2025-tmedina7/pull/8/commits/3bd5c67055690

c0355a8c20a5f3f218dfab9aff0

- **Branch created:** https://github.com/tymedina100/CSE464-2025-tmedina7/tree/refactor
- **Merge completed:** https://github.com/tymedina100/CSE464-2025-tmedina7/commit/32d9aea78903ebcb155e7f1e72c8e0d3bddc8075

Build Instructions

Build

- mvn package

Run Instructions

Common Inputs

- **Graph file:** src/main/resources/samples/demo_input.dot

BFS

Command:

- mvn -q exec:java -Dexec.mainClass=edu.asu.tmedina7.graph.Part3Demo

Expected output (screenshot below):

- Prints each visited path as it is explored (visiting Path{...}), then prints the final BFS path from a to h.

```
BFS demo: a -> h
visiting Path{nodes=[Node{a}]}
visiting Path{nodes=[Node{a}, Node{b}]}
visiting Path{nodes=[Node{a}, Node{e}]}
visiting Path{nodes=[Node{a}, Node{b}, Node{c}]}
visiting Path{nodes=[Node{a}, Node{e}, Node{f}]}
visiting Path{nodes=[Node{a}, Node{e}, Node{g}]}
visiting Path{nodes=[Node{a}, Node{b}, Node{c}, Node{d}]}
visiting Path{nodes=[Node{a}, Node{e}, Node{f}, Node{h}]}
Path{nodes=[Node{a}, Node{e}, Node{f}, Node{h}]}
```

DFS

Command:

- mvn -q exec:java -Dexec.mainClass=edu.asu.tmedina7.graph.Part3Demo

Expected output (screenshot below):

- Prints each visited path as it is explored (visiting Path{...}), then prints the final DFS path from a to h.

```
DFS demo: a -> h
visiting Path{nodes=[Node{a}]}
visiting Path{nodes=[Node{a}, Node{e}]}
visiting Path{nodes=[Node{a}, Node{e}, Node{g}]}
visiting Path{nodes=[Node{a}, Node{e}, Node{g}, Node{h}]}
Path{nodes=[Node{a}, Node{e}, Node{g}, Node{h}]}
```

Random Walk

Command:

- mvn -q exec:java -Dexec.mainClass=edu.asu.tmedina7.graph.Part3Demo

Expected output:

- Prints each visited path as it is explored for each random attempt (random testing #N + visiting Path{...}), then prints the final random-walk path from a to h; across runs the trace and final path can differ, demonstrating randomness.

```
random testing #1
visiting Path{nodes=[Node{a}]}
visiting Path{nodes=[Node{a}, Node{e}]}
visiting Path{nodes=[Node{a}, Node{e}, Node{g}]}
visiting Path{nodes=[Node{a}, Node{e}, Node{g}, Node{h}]}
Path{nodes=[Node{a}, Node{e}, Node{g}, Node{h}]}

random testing #2
visiting Path{nodes=[Node{a}]}
visiting Path{nodes=[Node{a}, Node{b}]}
visiting Path{nodes=[Node{a}, Node{b}, Node{c}]}
visiting Path{nodes=[Node{a}, Node{b}, Node{c}, Node{d}]}
visiting Path{nodes=[Node{a}, Node{e}]}
visiting Path{nodes=[Node{a}, Node{e}, Node{f}]}
visiting Path{nodes=[Node{a}, Node{e}, Node{f}, Node{h}]}
Path{nodes=[Node{a}, Node{e}, Node{f}, Node{h}]}

random testing #3
visiting Path{nodes=[Node{a}]}
visiting Path{nodes=[Node{a}, Node{e}]}
visiting Path{nodes=[Node{a}, Node{e}, Node{f}]}
visiting Path{nodes=[Node{a}, Node{e}, Node{f}, Node{h}]}
Path{nodes=[Node{a}, Node{e}, Node{f}, Node{h}]}
```

Required Demonstrations

Canvas Graph File Check (a → c)

Graph file: src/main/resources/samples/demo_input.dot

Command:

- mvn -q exec:java -Dexec.mainClass=edu.asu.tmedina7.graph.Part3Demo

```
Canvas graph demo: a -> c
visiting Path{nodes=[Node{a}]}
visiting Path{nodes=[Node{a}, Node{b}]}
visiting Path{nodes=[Node{a}, Node{e}]}
visiting Path{nodes=[Node{a}, Node{b}, Node{c}]}
Path{nodes=[Node{a}, Node{b}, Node{c}]}
```

Refactorings (5)

Each refactoring should have its own commit link above and should be a focused change.

Refactoring #1 — Rename internal identifiers for clarity

- **Why:** Reduce cognitive load and ambiguity by using names that match intent (domain terms + role), making the search logic easier to read, review, and maintain without changing behavior.

- **Commit:** https://github.com/tymedina100/CSE464-2025-tmedina7/pull/8/commits/3698983db8daeb844329f67fe8b9244ae3115224

Refactoring #2 — Extract methods for search precheck and path reconstruction

- **Why:** Shrink an oversized method, isolate repeated/independent steps, and make the core algorithm easier to follow. Also enables focused unit testing of preconditions and path reconstruction separately from traversal.

- **Commit:** https://github.com/tymedina100/CSE464-2025-tmedina7/pull/8/commits/46ec1204d45e396b4ed7cbd7d0fb2f2b1a109e88

Refactoring #3 — Introduce constants to remove magic strings/numbers

- **Why:** Improve correctness and consistency by centralizing repeated literals, preventing typos/inconsistent values, and making later changes (format strings, default limits, etc.) a single edit instead of a hunt.

- **Commit:** https://github.com/tymedina100/CSE464-2025-tmedina7/pull/8/commits/90de949d531e077e16a2f99283039bcb6c70345c

Refactoring #4 — Encapsulate graph field behind private accessor

- **Why:** Protect invariants and reduce coupling by hiding the internal representation. This keeps future changes to how the graph is stored/initialized from rippling through the class.

- **Commit:** https://github.com/tymedina100/CSE464-2025-tmedina7/pull/8/commits/dfb93a28bec8335d101d8dc9b9a5fce2f8aa348f

Refactoring #5 — Extract GraphIO (DOT parsing/export + Graphviz rendering)

- **Why:** Enforce separation of concerns: graph I/O/rendering is not search logic. Moving it out reduces responsibilities in GraphService, improves organization, and makes both search and I/O code easier to test and modify independently.

- **Commit:** https://github.com/tymedina100/CSE464-2025-tmedina7/pull/8/commits/bb0066ffc6e88119667d052ddd032796fe3773f1

Design Patterns

Template Method (BFS/DFS)

The shared BFS/DFS logic is implemented once in GraphSearchTemplate.search(srcLabel, dstLabel), which validates inputs, initializes frontier/visited/parent, loops by popping the next node, exploring outgoing neighbors, recording parents, and reconstructing the final Path. The only variation is the frontier policy, expressed as hook methods createFrontier, addToFrontier, and popFromFrontier.

BfsSearch overrides these hooks to behave like a FIFO queue (addLast/removeFirst), while DfsSearch overrides them to behave like a LIFO stack (push/pop).

Strategy (algo parameter)

GraphSearchStrategy defines a single operation, Path search(String srcLabel, String dstLabel). GraphService.GraphSearch(String srcLabel, String dstLabel, Algorithm algo) selects the concrete strategy at runtime by looking up algo in a Map<Algorithm, GraphSearchStrategy> (e.g., BFS -> bfsSearcher, DFS -> dfsSearcher) and then delegates with return strategy.search(srcLabel, dstLabel). BFS and DFS implement the choices by providing different strategy objects (BfsSearch and DfsSearch), both implementing GraphSearchStrategy (via GraphSearchTemplate implements GraphSearchStrategy).

CI / Workflow Evidence




